

Examinationsuppgift:

Du tar över ett befintligt C#-projekt för hantering av bokningar i en datorsal. Systemet är delvis färdigbyggt men innehåller fel, ofullständig funktionalitet och bristande felhantering.

Din uppgift är att självständigt **felsöka, korrigera och färdigställa systemet** så att det uppfyller följande krav:

- Projektet ska kunna byggas och startas utan kompileringsfel.
- Samtliga bokningar ska kunna visas i kronologisk ordning.
- Det ska gå att skapa en bokning med namn, starttid, sluttid och ändamål.
- Det ska gå att avboka och editera en bokning med hjälp av bokningens id.
- Överlappande bokningar ska förhindras.
- Ogiltiga tider ska avvisas.
- Felaktig eller tom användarinmatning ska hanteras utan att programmet kraschar.
- Koden ska vara uppdelad i klasser och metoder med tydliga ansvarsområden.

Två bokningar får ligga direkt efter varandra, exempelvis **10.00–11.00 och 11.00–12.00**.

Betygskriterier från kursplan:

Examinationen bedömer främst din förmåga att:

- Felsöka och hantera fel på ett strukturerat sätt, exempelvis med breakpoints och undantagshantering.
- Använda versionshantering med Git.
- Välja och använda lämpliga datastrukturer.
- Implementera programlogik med variabler, datatyper, operatorer, villkor och loopar.
- Använda objektorienterade principer och strukturera programmet med klasser, objekt och metoder.

För **Väl godkänt** bedöms även hur självständigt och metodiskt du arbetar samt hur väl du identifierar grundorsaker till problem och förbättrar kodens robusthet och struktur.

Betygsnivåer:

IG – Icke godkänt

Projektet kan inte byggas, centrala obligatoriska funktioner saknas eller fungerar inte, eller lösningen visar inte tillräcklig förmåga inom felsökning, felhantering och kodstruktur.

G – Godkänt

Samtliga obligatoriska funktioner fungerar. Överlappning, validering och felaktig

inmatning hanteras stabilt. Lösningen använder tydlig programlogik, lämpliga datastrukturer och en begriplig objektorienterad struktur.

VG – Väl godkänt

Samtliga krav för G är uppfyllda. Lösningen håller dessutom en jämn och robust kvalitet även i gränsfall. Du arbetar självständigt, identifierar grundorsaker till problem och förbättrar kodens struktur och felhantering med tydliga ansvarsområden och färre sidoeffekter.

Instruktioner:

1. Ladda ner startprojektet när examinationen börjar.
2. Skapa ett eget Git-repository och pusha startprojektet innan du börjar göra ändringar.
3. Bjud in läraren till repot.
4. Öppna lösningen och undersök vilka projekt, klasser och tester som finns.
5. Kör programmet och testa innan du börjar ändra koden.
6. Prioritera först att projektet bygger och att de obligatoriska funktionerna fungerar.
7. Använd debugger, breakpoints, stegning och relevanta felmeddelanden vid felsökning.
8. Gör små och tydliga ändringar och committa regelbundet.
9. Testa samtliga funktioner och relevanta gränsfall.
10. Lämna in repot innan examinationstiden är slut.

Tid: 4 timmar

Arbetsform: Individuellt arbete

Tillåtna verktyg: Git, Visual Studio eller Rider, debugger, terminal samt verktyg som ingår i startprojektet.

Generativ AI: Inte tillåtet.

Kommunikation: Ingen kommunikation eller kodelning med andra studenter under examinationen.

Du ska även lämna in en kort `REFLEKTION.md` eller `REFLEKTION.txt` där du beskriver vilka fel du identifierade, vilka ändringar du gjorde och hur du testade lösningen.